Facultatea de Matematică și Informatică
Universitatea din București



Programare orientată pe obiecte

- suport de curs -

ANCA DOBROVĂȚ
ANDREI PĂUN

AN UNIVERSITAR 2025 - 2026

Semestrul I

Seriile 21 și 26

CURS 6



Agenda cursului

1. Proiectarea descendentă a claselor. Moștenirea în C++ (recapitulare și completări la cursul 5)

   Modificatori de acces, constructori / destructori, redefinirea membrilor unei clase de bază într-o clasă derivată.
   Mostenire din baza multipla. Moștenire in diamant

2. Polimorfism la execuție prin funcții virtuale în C++.
   Parametrizarea metodelor (polimorfism la execuție).
   Funcții virtuale în C++.
   Clase abstracte.
   Overloading pe funcții virtuale
   Destructori și virtualizare



Moștenire multiplă (MM)

- puține limbaje au MM;
- moștenirea multiplă e complicată: ambiguitate LA MOȘTENIREA ÎN ROMB / IN DIAMANT;
- nu e nevoie de MM (se simulează cu moștenire simplă);
- se moșteneste în același timp din mai multe clase;

Sintaxa:
class Clasa_Derivată : [modificatori de acces] Clasa_de_Bază1, [modificatori de acces] Clasa_de_Bază2, [modificatori de acces] Clasa_de_Bază3 .....



Moștenire multiplă (MM)

Exemplu:

```
class Imprimanta {  };
class Scaner {  };
class Multifuncționala: public Imprimanta, public Scaner {   };
```

Ce ar putea crea probleme în cazul următor?

```
class Baza{  };
class Derivata_1: public Baza{  };
class Derivata_2: public Baza{  };
class Derivata_3: public Derivata_1, public Derivata_2 {  };
```

În **Derivata_3** avem de două ori variabilele din **Baza**!!



Moștenire multiplă : ambiguități (problema diamantului)

```
class base {   public:       int i; };
class derived1 : public base {  public:       int j; };
class derived2 : public base {   public:       int k; };
class derived3 : public derived1, public derived2 {public:     int sum; };

int main() {
derived3 ob;
ob.i = 10; // this is ambiguous, which i        // expl ob.derived1::i
ob.j = 20;
ob.k = 30;
ob.sum = ob.i + ob.j + ob.k; // i ambiguous here, too
cout << ob.i << " "; // also ambiguous, which i?
cout << ob.j << " " << ob.k << " ";
cout << ob.sum;
return 0;
}
```



Moștenire multiplă (MM)

- dar dacă avem nevoie doar de o copie a lui i?

- nu vrem să consumăm spațiu în memorie;

- ***folosim moștenire virtuală***:

```
class base {   public:      int i; };
class derived1 : virtual public base {  public:      int j; };
class derived2 : virtual public base {   public:      int k; };
class derived3 : public derived1, public derived2 {public:     int sum; };
```

- Dacă avem moștenire de două sau mai multe ori dintr-o clasă de bază (fiecare moștenire trebuie să fie virtuală) atunci compilatorul alocă spațiu pentru o singură copie;
- În clasele derived1 și 2 moștenirea e la fel ca mai înainte (niciun efect pentru virtual în acel caz)
-



2. Polimorfismul la execuţie prin funcţii virtuale

Funcţii virtuale

Funcțiile virtuale și felul lor de folosire: componentă IMPORTANTĂ a paradigmei OOP.

Folosit pentru polimorfism la execuţie ---> cod mai bine organizat cu polimorfism.

Codul poate “crește” fără schimbări semnificative: programe **ușor** de extins și întreținut.

Funcțiile virtuale sunt definite în bază și redefinite în clasa derivată.

Pointer de tip bază care arată către obiect de tip derivat și cheamă o funcție virtuală în bază și redefinite în clasa derivată execută ***Funcția din clasa derivată***.

Poate fi văzută ca exemplu de separare dintre interfață și implementare.



2. Polimorfismul la execuție prin funcții virtuale

Decuplare în privința tipurilor

Upcasting - Tipul derivat poate lua locul tipului de bază (foarte important pentru procesarea mai multor tipuri prin același cod).

Funcții virtuale: ne lasă să chemăm funcțiile pentru tipul derivat.

Problemă: apel la funcție prin pointer (tipul pointerului ne dă funcția apelată).



2. Polimorfismul la execuție prin funcții virtuale

```
enum note { middleC, Csharp, Eflat }; // Etc.

class Instrument {  public:
  virtual void play(note) const = 0;
};
void Instrument::play(note) const {  std::cout << "Instrument::play" << std::endl;  }

class Wind : public Instrument {
public:  void play(note) const override {  std::cout << "Wind::play" << std::endl; }};

class String : public Instrument {
public:  void play(note) const override {  std::cout << "String::play" << std::endl; }};

void tune(Instrument& i) {  i.play(middleC);  }

int main() {
  Wind flute;  String cello;
  tune(flute);  tune(cello);
}
```



2. Polimorfismul la execuție prin funcții virtuale

În C ---> early binding la apel de funcții - se face la compilare (dar... tinde spre comportament dinamic).

În C++ ---> putem defini late binding prin funcții virtuale (late, dynamic, runtime binding) - se face apel de funcție bazat pe tipul obiectului, la rulare (nu se poate face la compilare). Dar… tinde spre mai multe operații la compilare.

Late binding ===> prin pointeri!

Late binding pentru o funcție: se scrie virtual inainte de definirea funcției.

Pentru clasa de bază: nu se schimbă nimic!

Pentru clasa derivată: late binding însemnă că un obiect derivat folosit în locul obiectului de bază își va folosi funcția sa, nu cea din bază (din cauză de late binding).

Utilitate: putem extinde codul precedent fara schimbari in codul deja scris.



Cum se face late binding

Tipul obiectului este ținut în obiect pentru clasele cu funcții virtuale.

Late binding se face (uzual) cu o tabelă de pointeri: vptr către funcții.

În tabelă sunt adresele funcțiilor virtuale ale clasei respective (fie adrese din bază, fie din derivate).

Fiecare obiect din clasă are pointerul acesta în componență.

La apel de funcție membru se merge la obiect, se apelează funcția prin vptr.

Vptr este inițializat în constructor (automat) cu adresa cea mai derivată corespunzătoare clasei obiectului creat.



2. Polimorfismul la execuţie prin funcţii virtuale

Cum se face late binding

```
class NoVirtual {  int a;
public:
  void x() const {}
  int i() const { return 1; } };

class OneVirtual {  int a;
public:
  virtual void x() const {}
  int i() const { return 1; } };

class TwoVirtuals {  int a;
public:
  virtual void x() const {}
  virtual int i() const { return 1; } };
```

```
int main() {
  cout << "int: " << sizeof(int) << endl;
  cout << "NoVirtual: "
       << sizeof(NoVirtual) << endl;
  cout << "void* : " << sizeof(void*) << endl;
  cout << "OneVirtual: "
       << sizeof(OneVirtual) << endl;
  cout << "TwoVirtuals: "
       << sizeof(TwoVirtuals) << endl;
}
```



Cum se face late binding

```
class Pet { public:
  virtual string speak() const { return " "; } };

class Dog : public Pet { public:
  string speak() const { return "Bark!"; } };

int main() {
  Dog ralph;
  Pet* p1 = &ralph;
  Pet& p2 = ralph;
  Pet p3;
  // Late binding for both:
  cout << "p1->speak() = " << p1->speak() <<endl;
  cout << "p2.speak() = " << p2.speak() << endl;
  // Early binding (probably):
  cout << "p3.speak() = " << p3.speak() << endl;
}
```



2. Polimorfismul la execuție prin funcții virtuale

Dacă funcțiile virtuale sunt așa de importante, de ce nu sunt toate funcțiile definite virtuale (din oficiu)?

Deoarece “costă” în memoria (și poate viteza) programului.

În Java sunt doar virtuale, dar Java ocupă mult mai multă memorie.

Nu mai putem avea funcții inline* (ne trebuie adresa funcției pentru **vptr**).



Clase abstracte și funcții virtuale pure

Clasă abstractă = clasă care are cel puțin o funcție virtuală PURĂ

Necesitate: clase care dau doar interfață (nu vrem obiecte din clasă abstractă ci upcasting la ea).

Eroare la instanțierea unei clase abstracte (nu se pot defini obiecte de tipul respectiv).

Permisă utilizarea de pointeri și referințe către clasă abstractă (pentru upcasting).

Nu pot fi trimise către funcții (prin valoare).



2. Polimorfismul la execuţie prin funcţii virtuale

Funcții virtuale pure

Sintaxa: **virtual** ***tip_returnat nume_funcţie(lista_parametri) =0;***

Ex: virtual int pura(int i)=0;

Obs: La moștenire, dacă în clasa derivată nu se definește funcția pură, clasa derivată este și ea clasă abstractă ---> nu trebuie definită funcție care nu se execută niciodată

UTILIZARE IMPORTANTĂ: prevenirea “object slicing”.



2. Polimorfismul la execuție prin funcții virtuale

Clase abstracte și funcții virtuale pure

```
class Pet {  string pname;
public:
  Pet(const string& name) : pname(name) {}
  virtual string name() const { return pname; }
  virtual string description() const {
    return "This is " + pname;
  }
};

class Dog : public Pet {  string favoriteActivity;
public:
  Dog(const string& name, const string& activity)
    : Pet(name), favoriteActivity(activity) {}
  string description() const {
    return Pet::name() + " likes to " + favoriteActivity;
  }
};
```

```
void describe(Pet p) { // Slicing
  cout << p.description() << endl;
}

int main() {
  Pet p("Alfred");
  Dog d("Fluffy", "sleep");
  describe(p);
  describe(d);
}
```



Overload pe funcții virtuale

Obs. Nu e posibil overload prin schimbarea tipului param. de întoarcere (e posibil pentru ne-virtuale)

De ce. Pentru că se vrea să se garanteze că se poate chema baza prin apelul respectiv.

Excepție: tipuri de date covariante - pointer/referință către bază întors în bază, pointer/referință către derivată în derivată



2. Polimorfismul la execuţie prin funcţii virtuale

Overload pe funcţii virtuale

```
class Base {
public:
  virtual int f() const {
    cout << "Base::f()\n";  return 1;   }
  virtual void f(string) const {}
  virtual void g() const {}
};

class Derived1 : public Base {public:
  void g() const {}
};

class Derived2 : public Base {public:
  // Overriding a virtual function:
  int f() const {  cout << "Derived2::f()\n";
    return 2;  }
};
```

```
int main() {
  string s("hello");
  Derived1 d1;
  int x = d1.f();
  d1.f(s);
  Derived2 d2;
  x = d2.f();
//!  d2.f(s); // string version hidden
}
```



2. Polimorfismul la execuţie prin funcţii virtuale

Overload pe funcţii virtuale

```
class Base {
public:
  virtual int f() const {
    cout << "Base::f()\n";  return 1;   }
  virtual void f(string) const {}
  virtual void g() const {}
};

class Derived3 : public Base {public:
//! void f() const{ cout << "Derived3::f()\n";}};

class Derived4 : public Base {public:
  // Change argument list:
  int f(int) const
        { cout << “Derived4::f()\n"; return 4; }
};
```

```
int main() {
  string s("hello");
  Derived4 d4;
  x = d4.f(1);
//!  x = d4.f(); // f() version hidden
//!  d4.f(s); // string version hidden
  Base& br = d4; // Upcast
//!  br.f(1); // Derived version
unavailable
  br.f();
  br.f(s); // Base version available
return 0;
}
```



Constructori și virtualizare

Obs. NU putem avea constructori virtuali.

În general pentru funcțiile virtuale se utilizează late binding, dar în utilizarea funcțiilor virtuale în constructori, varianta locală este folosită (early binding)

De ce?

Pentru că funcția virtuală din clasa derivată ar putea crede că obiectul e inițializat deja

Pentru că la nivel de compilator în acel moment doar VPTR local este cunoscut



Constructori și virtualizare

Obs. NU putem avea constructori virtuali (în sensul "virtual Instrument() {}")

NU apelăm funcții virtuale în constructori sau destructori (detalii)

- Se va apela definiția din clasa curentă, nu dintr-o clasă mai derivată
- Deși de obicei nu este o problemă în sine, poate cauza confuzie în proiecte mari
- În limbaje dinamice (e.g. Python) nu este o problemă, dar acolo nu avem verificări de tipuri prea stricte

Soluții:

- Constructori virtuali (funcții de clonare)
- Clase și funcții de tip factory

Dacă avem nevoie de funcții virtuale în destructori, ar trebui să ne întrebăm de ce am avea nevoie de așa ceva

- "soluție" (sau soluție când nu avem de ales): apel explicit: Base::f() sau Derived::f()



Destructori și virtualizare

Se cheamă în ordine inversă decât constructorii.

În general două situații:

- Destructor public și virtual
  - Dacă avem și alte funcții virtuale, deci folosim pointeri către bază



Destructori și virtualizare

```
class Base1 {public:  ~Base1() { cout << "~Base1()\n"; } };

class Derived1 : public Base1 {public:  ~Derived1() { cout << "~Derived1()\n"; } };

class Base2 {public:
  virtual ~Base2() { cout << "~Base2()\n"; }
};

class Derived2 : public Base2 {public:  ~Derived2() { cout << "~Derived2()\n"; } };

int main() {
  Base1* bp = new Derived1;
  delete bp; // Afis: ~Base1()
  Base2* b2p = new Derived2;
  delete b2p; // Afis: ~Derived2()  ~Base2()
}
```



Destructori virtuali puri

Utilizare: poate fi utilizat dacă avem funcții virtuale pe care nu vrem să le suprascriem în toate derivatele.

Restricție: trebuie să aibă o definiție (chiar dacă este abstractă).

La moștenire nu mai trebuie redefiniți (se construiește un destructor din oficiu)

De ce? Pentru a preveni instanțierea clasei.

Obs. Nu are nici un efect dacă nu se face upcasting, dar atunci am folosi destructor protected și non-virtual.

```
class AbstractBase {
public:
  virtual ~AbstractBase() = 0;
};

AbstractBase::~AbstractBase() {}

class Derived : public AbstractBase {};
// No overriding of destructor necessary?
int main() { Derived d; }
```



Funcții virtuale în destructori

La apel de funcție virtuală din funcții normale se apelează conform VPTR
În destructori se face early binding! (apeluri locale)
De ce? Pentru că acel apel poate să se bazeze pe porțiuni deja distruse din obiect

```
class Base { public:
  virtual ~Base() {  cout << "~Base1()\n";  this->f();   }
  virtual void f() { cout << "Base::f()\n"; }
};
class Derived : public Base { public:
  ~Derived() { cout << "~Derived()\n"; }
  void f() { cout << "Derived::f()\n"; }
};

int main() {
  Base* bp = new Derived;
  delete bp; // Afis: ~Derived() ~Base1() Base::f()
}
```



Downcasting

Folosit in ierarhii polimorfice (cu funcţii virtuale) pentru a obține înapoi derivata către care arată un pointer de tip bază.

Problema: upcasting e sigur pentru că respectivele funcții/atribute trebuie să fie definite în bază, downcasting e problematic – nu știm ce avem de fapt.

Explicit cast prin: **dynamic_cast**

Dacă știm cu siguranţă tipul obiectului, putem folosi "static_cast".
static_cast întoarce pointer către obiectul care satisface cerințele sau 0.

dynamic_cast folosește tabelele VTABLE pentru determinarea tipului.

Conversiile tip C++ documentează de ce avem nevoie de un anumit cast
Numele conversiilor sunt lungi pentru a atrage atenția (și pentru a le evita).



Downcasting

```
class Pet { public: virtual ~Pet(){}};
class Dog : public Pet {};
class Cat : public Pet {};

int main() {
  Pet* b = new Cat; // Upcast
  Dog* d1 = dynamic_cast<Dog*>(b);  // Afis - 0; Try to cast it to Dog*:
  Cat* d2 = dynamic_cast<Cat*>(b);// Try to cast it to Cat*:
  // b si d2 retin aceeasi adresa
  cout << "d1 = " << d1 << endl;
  cout << "d2 = " << d2 << endl;
cout << "b  = " << b << endl;
}
```



2. Downcasting

typeid (sau atribut intern) + static_cast

```
class Shape { public: virtual ~Shape() {}; };
class Circle : public Shape {};
class Square : public Shape {};
class Other {};

int main() {
  Circle c;
  Shape* s = &c; // Upcast: normal and OK
  // More explicit but unnecessary:
  s = static_cast<Shape*>(&c);
  // (Since upcasting is such a safe and common
  // operation, the cast becomes cluttering)
  Circle* cp = 0;
  Square* sp = 0;
  // Static Navigation of class hierarchies
requires extra type information:
  if(s && typeid(*s) == typeid(Circle)) //
C++ RTTI
    cp = static_cast<Circle*>(s);
  if(s && typeid(*s) == typeid(Square))
    sp = static_cast<Square*>(s);
  if(cp != 0)
    cout << "It's a circle!" << endl;
  if(sp != 0)
    cout << "It's a square!" << endl;
  // Static navigation is ONLY an efficiency
hack;
  // dynamic_cast is always safer. However:
  // Other* op = static_cast<Other*>(s);
  // Conveniently gives an error message,
while
  Other* op2 = (Other*)s;
  // does not
}
```



PERSPECTIVE

Cursul 7:

Moștenire – completări:

Copy constructor, operator =, destructor pt clasele cu atribute de tip pointer
- clone, copy&swap,
- RAII (*Resource Acquisition Is Initialization*)

Tratarea excepțiilor